

High Performance Computing – Week 2 Task

Orazgali Azamat, BDA-2402

Part 1: Memory Hierarchy

1. Arrange from fastest to slowest:

Registers, L1 Cache, L2 Cache, Main Memory, Local Disk.

2. What is a cache hit?

A cache hit happens when the data the processor asks for is already in the cache, so it can be read quickly without going to slower memory

3. What is a cache miss?

A cache miss happens when the requested data is not in the cache. It has to be loaded from a slower level (for example from L2, main memory, etc.), which takes more time

4. Difference between temporal and spatial locality:

a) Temporal locality: if a piece of data was used recently, it is likely to be used again soon (for example a loop variable or the sum)

b) Spatial locality: if a memory location was accessed, locations close to it (its neighbours) are likely to be accessed soon (for example neighbour elements)

5. Which locality is used when accessing consecutive array elements?

Spatial locality, because we bring a block of consecutive data from memory to cache the consecutive array elements due to their neighbor locations

Part 2: Matrix Access Experiment

Experiment A: Row-wise access

Run	Time (seconds)
1	0.0043189
2	0.0030766
3	0.0029402

Experiment B: Column-wise access

Run	Time (seconds)
1	0.0138046
2	0.0138076
3	0.0155111

Two more runs of each gave 0.0030709 and 0.0032214 (row-wise), and 0.0169070 and 0.0145319 (column-wise), so the results are consistent.

Questions

1. Do both versions calculate the same result?

Yes. Both print Sum = 16000000, because they add the same 16,000,000 elements, only in a different order of dimensions

2. Which version was faster on your computer?

The row-wise version. It took about 0.003 s, while the column-wise version took about 0.014–0.015 s, so it was roughly 4–5 times faster

3. In the row-wise version, are consecutive memory locations accessed frequently?

Yes. The inner loop goes over j , so $A[i * N + j]$ and $A[i * N + j + 1]$ are neighbors in memory and are read one after another

4. Which version has better spatial locality?

The row-wise version

5. Why can changing the loop order affect time even though the number of additions is the same?

The matrix is stored row by row. In the column-wise loop, each next access jumps $N = 4000$ integers (16000 bytes) ahead, so it lands in a different cache line every time. Most of the loaded cache line is not used before it is evicted, which causes many cache misses and slow main-memory accesses. In the row-wise loop one cache line supplies several consecutive elements, so most accesses are cache hits

Part 3: Simple Parallel Program Design

1. Partitioning

Processor	First Row	Last Row
P0	0	999
P1	1000	1999
P2	2000	2999
P3	3000	3999

1. Can the four groups of rows be processed independently?

Yes. Each element is updated using only its own value, so no group depends on another.

2. Domain decomposition or functional decomposition?

Domain decomposition, because we split the matrix and every processor does the same operation on its part

2. Communication

1. Does P0 need data from P1?

No

2. Do neighboring processors need to exchange matrix elements?

No, each processor only works with its own rows

3. Type of communication:

No communication

3. Agglomeration

1. One task per matrix element: fine-grained or coarse-grained?

Fine-grained parallelism

2. Why combine many small tasks into larger ones?

Creating, scheduling and managing 16,000,000 tiny tasks costs more time than the work of one addition. Larger tasks reduce this overhead (and possible communication/synchronization), and also keep memory access contiguous, which is good for the cache.

3. Reasonable larger task size for four processors:

One task per processor: a block of 1000 consecutive rows ($1000 \times 4000 = 4,000,000$ elements).

4. Mapping

Task	Processor
Rows 0–999	P0
Rows 1000–1999	P1
Rows 2000–2999	P2
Rows 3000–3999	P3

1. Is the amount of work approximately balanced?

Yes. Every processor gets 1000 rows (4,000,000 elements), and the processors have the same performance.

2. What could happen if one processor got 3000 rows and the others about 333 rows?

The load would be unbalanced. The processors with 333 rows would finish quickly and then sit idle while the one with 3000 rows keeps working. The total time is determined by the slowest processor, so it would be about 3 times longer than with a balanced split and the speedup would be poor (only about 1.33 compared to one processor instead of 4)

Part 4: Reduction

$$S = S0 + S1 + S2 + S3$$

1. Is combining S0, S1, S2, S3 a local or global operation?

Global operation, because it needs values from all processors

2. What is this operation commonly called?

A reduction (here a sum reduction).

Part 5: Short Analysis

1. Do the programs have the same asymptotic time complexity?

Yes

2. Time complexity in Big-O:

$O(N^2)$, since both do $N \times N$ increments

3. Can actual execution times be different?

Yes. Big-O ignores the cost of each memory access, so the real times can differ a lot

4. Which hardware feature explains the difference?

The cache (memory hierarchy) and spatial locality. Program A accesses memory sequentially and gets mostly cache hits; Program B jumps by N elements and gets many cache misses.